16 · Route Parameters, Query Params and Child Routes

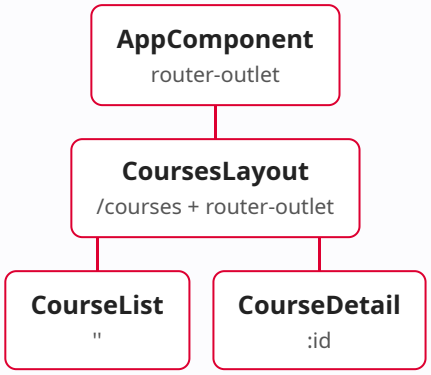
Intermediate Angular learning notes

a. Definition

A **route parameter** is a changing part of the URL, written `:id` in the path (`/courses/2`). **Query parameters** come after `?` (`/courses/2?tab=syllabus`). With `withComponentInputBinding()` the router passes both straight into component `input()`s - no `ActivatedRoute` needed.

Child routes nest pages inside a parent layout: the parent has its own `<router-outlet />` and the `children` array decides what appears inside it.

b. Diagram



/courses/2?tab=syllabus -> CoursesLayout -> CourseDetail (id=2, tab=syllabus)

c. Syntax

```
{ path: 'courses', component: CoursesLayout, children: [
    { path: '',    component: CourseList },
    { path: ':id', component: CourseDetail },
] }

provideRouter(routes, withComponentInputBinding())

id  = input.required<string>();     // from :id
tab = input('overview');            // from ?tab=...

<a [routerLink]="['/courses', 2]" [queryParams]="{ tab: 'syllabus' }">
```

d. Example Program

`Programs/16_Route_Params_and_Child_Routes/courses-router.component.ts`

```
import { ApplicationConfig, Component, computed, input } from '@angular/core';
import { provideRouter, RouterLink, RouterOutlet, Routes, withComponentInputBinding,
withHashLocation } from '@angular/router';
```

```ts
const COURSES = [
  { id: 1, title: 'Angular Basics' },
  { id: 2, title: 'TypeScript Deep Dive' },
];

// Parent layout: its own <router-outlet> shows the child routes
@Component({
  selector: 'app-courses-layout',
  imports: [RouterOutlet],
  template: `<h2>Courses</h2><router-outlet />`,
})
export class CoursesLayoutComponent {}

@Component({
  selector: 'app-course-list',
  imports: [RouterLink],
  template: `
    <ul>
      @for (c of courses; track c.id) {
        <li><a [routerLink]="[c.id]" [queryParams]="{ tab: 'syllabus' }">{{ c.title }}</a>
</li>
      }
    </ul>
  `,
})
export class CourseListComponent {
  courses = COURSES;
}

@Component({
  selector: 'app-course-detail',
  imports: [RouterLink],
  template: `
    <h3>Course #{{ id() }}: {{ course()?.title ?? 'Unknown' }}</h3>
    <p>Showing tab: {{ tab() }}</p>
    <a routerLink="/courses">Back to list</a>
  `,
})
export class CourseDetailComponent {
  // Filled from the URL thanks to withComponentInputBinding()
  id = input.required<string>();   // path param  :id
  tab = input('overview');         // query param ?tab=
  course = computed(() => COURSES.find((c) => c.id === Number(this.id())));
}

const routes: Routes = [
  { path: '', redirectTo: 'courses', pathMatch: 'full' },
  {
    path: 'courses',
    component: CoursesLayoutComponent,
    children: [
      { path: '', component: CourseListComponent },
      { path: ':id', component: CourseDetailComponent },
    ],
  },
];

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withComponentInputBinding(), withHashLocation())],
};

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  template: `<router-outlet />`,
})
export class AppComponent {}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. The list page opens first.
2. Click the second course.

```
--- Course list ---
Courses
Angular Basics
TypeScript Deep Dive

--- Course detail ---
Courses
Course #2: TypeScript Deep Dive
Showing tab: syllabus
Back to list
```

f. Real-World Example (English + தமிழ்)

English

A postal address 'Anna Nagar / house 12': the area is the layout and 12 is the changing part (parameter). A query parameter is a note like 'urgent'.

தமிழ்

தபால் முகவரியை எடுத்துக்கொள்ளுங்கள்: 'அண்ணா நகர் / 12-ஆம் எண் வீடு'. 'அண்ணா நகர்' என்பது பெற்றோர் (layout) - அந்தப் பகுதியின் பொதுவான அமைப்பு. '12' என்பது மாறக்கூடிய பகுதி (`:id` parameter) - ஒவ்வொரு வீட்டுக்கும் வேறு எண்.

கடைசியில் 'தபால் பெட்டி எண்: 5, அவசரம்' என்று கூடுதல் குறிப்பு எழுதுவது போல `?tab=syllabus` என்பது query parameter. முகவரி மாறினாலும் தபால்காரர் (router) சரியான வீட்டைக் கண்டுபிடிப்பார்.

g. Pros

- Shareable URLs
- Params bound directly to inputs
- Nested layouts via child routes

h. Cons

- Params are always strings
- Easy to forget outlet in parent routes

i. Advanced / New Features

- `withComponentInputBinding()` binds params, query and data to inputs
- Router state signals (`toSignal`)

j. Common Mistakes

- Forgetting `withComponentInputBinding()` ; inputs stay empty.
- Giving the parent route a component without a `<router-outlet />` ; children never show.
- Reading `id` as a number; route params are always strings (convert with `Number()`).
- Using absolute links inside children by accident, or relative links from the wrong route.

k. Interview Questions

Q1. Route params vs query params?
Route params are part of the path (`/courses/2` , required for matching); query params follow `?` (optional, e.g. filters).

Q2. How do you read a route parameter?
Via `input()` with `withComponentInputBinding()` , or `ActivatedRoute.paramMap` / `snapshot` .

Q3. What are child routes?
Routes nested under a parent route; the parent's `<router-outlet />` displays them.

Q4. What does `pathMatch: 'full'` mean?
The whole remaining URL must match the path, important for `''` redirects.

l. Practice Exercises

1. Add a `/products/:id` route and show the product's name and price from an array.
2. Add a `?sort=price` query parameter that sorts the course list.
3. Create a `/admin` layout with child routes `users` and `settings` .